Multimodal Visual Understanding（Text Version）

Before running the function, you need to close the App and large programs. For the closing method, refer to [4. Preparation] - [1. Manage APP control services].

1. Function Description

After the program runs, you can input visual-related questions or descriptions of the current scene through the terminal. The program will take an image of the current environment and upload it to the Alibaba Cloud platform. The visual model will then analyze the image based on the question and provide an answer.

2. Startup

After entering the Docker container, enter the following commands in the Docker terminal.

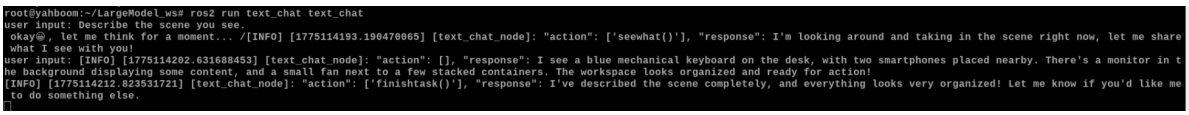
```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

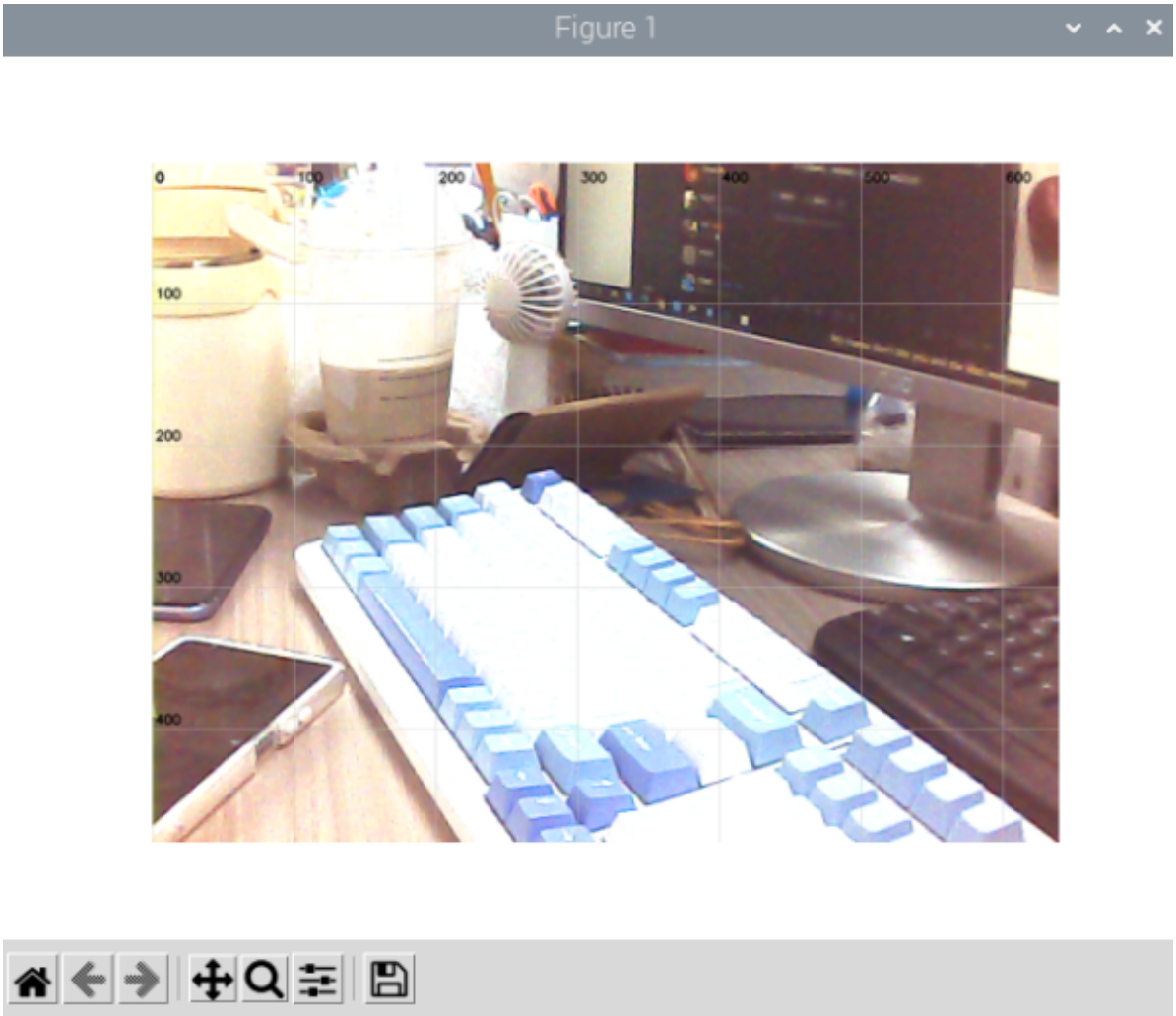
Then, in the text_chat terminal, enter your visual understanding-related questions. You can refer to the following examples:

Describe the scene you see.



The saved photo is located at:

`LargeModel_ws/install/largemodel/share/largemodel/resources_file/image.png`



3. Core Code Analysis: seewhat

The program calls the `seewhat` function to take photos. The source code path of this function is: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def seewhat(self):
    #Save image
    self.save_single_image()
    time.sleep(3.0)
    msg = String(data="seewhat")
    self.seewhat_handle_pub.publish(
        msg
    )  # Normalization, publishing the 'seewhat' topic, and calling the large language model via model_service.

def save_single_image(self):
    """
    保存一张图片 / Save a single image
    """
    self.IS_SAVING=True
    time.sleep(0.5)
    if self.image_msg is None:
        self.get_logger().warning("No image received yet.")  # Image not yet received...
        return
    try:
        # 将ROS图像消息转换为OpenCV图像 / Convert ROS image message to OpenCV image
        cv_image = self.bridge.imgmsg_to_cv2(self.image_msg, "bgr8")
```

```
        # 保存图片 / Save the image
        cv2.imwrite(self.image_save_path, cv_image)
        display_thread = threading.Thread(target=self.display_saved_image)
        display_thread.start()

    except Exception as e:
        self.get_logger().error(f"Error saving image: {e}")  # Error saving
image...
    self.IS_SAVING=False
```